CASE STUDY

The CVE-2026-98432 Linux Kernel Flaw and an AI-Written Exploit: A 2026 Case Study

A TeamStarWolf case study. Published 3 November 2026; every source checked as of that date.

Executive summary

In October 2026 the Linux kernel project disclosed CVE-2026-98432, a use-after-free in the IPv4 TCP connection-setup path that a remote, unauthenticated attacker can reach by sending crafted packets to a listening socket. The kernel CNA scored it 9.8, and the upstream fix is two lines. What made this flaw a landmark was not the bug, which is an ordinary memory-safety race of a kind the kernel fixes regularly, but what Anthropic published alongside it: its Frontier Red Team reported that an unreleased internal research model, after initial prompting, wrote a working zero-click remote-root exploit against servers running a Multipath TCP listener, reaching a root shell in under an hour of model time. The exploit itself was never released.

Two facts have to be held apart, and this study keeps them apart throughout. The vulnerability is Confirmed: the kernel project, the GitHub Advisory Database, Red Hat and AWS all describe the same use-after-free and the same `kfree_rcu` fix. The AI-written exploit is Reported: it rests on Anthropic's own account, the code and test configuration were deliberately withheld, and no independent public reproduction had appeared by late October. The significance for a vulnerability management program does not depend on resolving that second question. Whether or not this particular exploit works as described, the disclosure is a credible, dated marker that the time between a memory-corruption CVE landing and a working exploit existing is collapsing toward the length of a lunch break, which changes how fast the prioritization-to-patch loop has to turn.

> Evidence grade: the vulnerability is **Confirmed**; the AI-written exploit and its timing are **Reported** (single-source, Anthropic, unreleased and not independently reproduced as of this writing).

Key facts at a glance

Field	Value
Identifier	CVE-2026-98432 (GitHub advisory GHSA-qp6h-4hvh-qqqc)
Class	Use-after-free (CWE-416) from a race condition in IPv4 TCP connection handling
CVSS	9.8 Critical, vector CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H (kernel CNA, NVD, Red Hat)
EPSS	About 0.46 percent (roughly the 40th percentile) as of early November 2026
Access required	None: remote, unauthenticated, no user interaction
Affected	Linux kernel 4.4 and later, IPv4 only; the primary issue does not affect IPv6
Worst outcome	Remote code execution on hosts with a Multipath TCP listener; kernel crash, memory disclosure or corruption otherwise
Upstream fix	Two `kfree()` calls changed to `kfree_rcu()` ; landed 5 October 2026, backported to maintained stable branches
Disclosed	CVE published 15 October 2026; AI-exploit report published by Anthropic 28 October 2026
In the wild	No reported exploitation; not in CISA KEV as of 2 November 2026

Key findings

- **The bug is an unauthenticated remote use-after-free in IPv4 TCP.** CVE-2026-98432 (CVSS 9.8) is a race condition during concurrent processing of a retransmitted TCP connection request or a socket migration. A child socket is handed the request socket's IP options without a copy, and the options can be freed with a plain `kfree()` while the SYNACK path is still reading them under RCU, which is the use-after-free. IPv4 is affected; IPv6 is not affected by the primary issue.
- **The upstream fix is two lines.** The patch converts the two plain `kfree()` calls, in `inet_sock_destruct()` and `tcp_v4_reqsk_destructor()` , to `kfree_rcu()` , so the options are freed only after an RCU grace period, after readers are done. It landed upstream on 5 October 2026 and was backported to the maintained stable branches, and the CVE was published on 15 October 2026.
- **Impact depends on configuration.** On a host with a Multipath TCP listener, researchers reported unauthenticated remote code execution. Without that, the reachable outcome on plain IPv4 TCP is a kernel crash, memory disclosure, or memory corruption, which is a denial of service and a possible stepping stone. Red Hat notes that reliable code execution "requires precise heap control."
- **An AI model reportedly wrote the exploit, fast, with little human steering.** Anthropic's Frontier Red Team reported that an unreleased internal model wrote the MPTCP exploit after initial prompting, taking by

its own account "48 minutes to leak a kernel pointer and another five minutes to reach the shell command execution." Anthropic did not publish the exploit, the kernel version, or the configuration it tested, and framed the result as a lab exercise under coordinated disclosure, not an in-the-wild attack.

- **The ordinary triage signals still said "urgent, not on fire."** As of early November 2026 there was no reported exploitation in the wild, the CVE was not in CISA's Known Exploited Vulnerabilities catalog, and the EPSS score sat around 0.46 percent. A 9.8 base score with an AI-exploit headline and a low real-world-use signal is exactly the kind of split a prioritization program has to reconcile rather than resolve by the loudest number.

Top recommendations

1. Patch the kernel and reboot. Apply your distribution's fixed kernel as soon as it ships and reboot, because a running kernel keeps the vulnerable code until it is replaced. Upstream fixed this on 5 October 2026, but distribution availability lagged by weeks, so confirm your own vendor's status rather than assuming "upstream fixed" means you are fixed.
2. Find your Multipath TCP exposure first. Identify hosts with an MPTCP listener, because those carry the reported remote-code-execution risk rather than denial of service, and prioritize them. Note that disabling MPTCP alone is not a complete fix: the plain IPv4 TCP path remains reachable for the crash-and-corruption outcome.
3. Use the temporary network mitigation only as a bridge. Dropping inbound IPv4 TCP packets that carry IP options (for example with an nftables rule) blunts the common trigger, but vendors are explicit that it is not a substitute for patching, it needs testing, and on container hosts it may not cover pod-to-pod traffic.
4. Reconcile the base score with the real-world signal. Treat a 9.8 with no in-the-wild use, no KEV listing, and a low EPSS as a fast-track-but-verify item: schedule it ahead of routine patching, scope it by exposure (internet-facing, MPTCP), and do not let an AI-exploit headline push it past an actively exploited bug with a lower score.
5. Shorten your own exploit-window assumptions. Plan patch SLAs for critical memory-corruption CVEs on the assumption that a working exploit can exist within hours of disclosure, for defenders and attackers alike, and that scanning your own code and dependencies before an advisory lands is now part of the job.

Scope and method

This study covers CVE-2026-98432 as disclosed in October 2026 and the AI-exploit claim published alongside it. Sources are the kernel CVE record and GitHub Advisory Database, Red Hat, AWS and other distribution advisories, and Anthropic's own research post for the exploit claim. Each claim carries one of three evidence grades, and the AI-exploit material is held to the same bar as everything else.

Grade	Meaning
Confirmed	The kernel project, a distribution vendor, or a government advisory confirms the vulnerability or the fix
Reported	A single source's account, not independently corroborated or reproduced

Grade	Meaning
Demonstrated	A research reproduction available for others to check

Part 1: the vulnerability in detail

Where it lives

CVE-2026-98432 lives in the kernel's handling of incoming TCP connections (**Confirmed**: kernel CNA, GitHub Advisory GHSA-qp6h-4hvh-qqqc, Red Hat, AWS). To understand it you need three pieces of how Linux accepts a connection.

First, **IP options**. A TCP/IP packet can carry optional header fields, the IP options, which the sender controls. The kernel parses them into a small `ip_options` structure attached to the socket. Because the sender sets them, they are attacker-controlled input that the kernel keeps and later reads.

Second, **request sockets**. When a SYN arrives on a listening socket, the kernel does not immediately create a full socket. It creates a lightweight request socket to track the half-open connection, and only promotes it to a full child socket when the handshake completes. Since an earlier kernel change (commit `079096f103fa`), request sockets are processed without holding the listener lock, which improved scalability but removed the serialization that used to make the next step safe.

Third, **RCU**, read-copy-update, a kernel pattern where readers take a lightweight `rcu_read_lock()` and writers defer freeing memory until all such readers have finished, a moment called a grace period. RCU only protects a structure if everyone frees it the RCU way; a single plain `kfree()` on an RCU-read structure breaks the guarantee.

The race

When the handshake completes, `tcp_v4_syn_recv_sock()` builds the child socket and hands it the request socket's `ip_options` by pointer, without making a copy. At almost the same moment, another CPU can be processing a retransmitted SYN for the same request socket and sending a SYNACK. The SYNACK path reads the options while holding only `rcu_read_lock()`, and `ip_options_build()` reads `opt->optlen` twice as it constructs the reply. Because request sockets are no longer under the listener lock, and because TCP child sockets are not marked `SOCK_RCU_FREE`, the child can be destroyed in the middle of that read. On destruction, `inet_sock_destruct()` frees the options with a plain `kfree()`. The concurrent reader is now walking freed memory: a use-after-free, reachable from the network, before any application has seen a byte (**Confirmed**).

A second, related path exists for request-socket migration, used when `net.ipv4.tcp_migrate_req` is enabled or a `BPF_SK_REUSEPORT_SELECT_OR_MIGRATE` program is attached to move half-open connections between listeners (common in zero-downtime restarts of busy servers). The cloned request socket shares the same options pointer, and `tcp_v4_reqsk_destructor()` also frees it with a plain `kfree()`, giving a second way to trigger the same class of bug.

IPv6 is not affected by the primary issue, because `tcp_v6_syn_recv_sock()` duplicates the options rather than sharing them. The migration path has a narrower IPv6 variant tied to options that only CALIPSO sets, and the advisory says that is addressed in a separate patch.

The trigger and the blast radius

The trigger is a crafted IPv4 TCP packet carrying IP options, sent to a listening socket, with the timing arranged to race a retransmitted SYN; the migration variant additionally needs the migration feature turned on. No credentials and no user interaction are required.

What an attacker gets depends on the target (**Confirmed** for the outcomes, **Reported** for the demonstrated RCE):

- On a host with a **Multipath TCP (MPTCP) listener**, researchers reported a full unauthenticated remote code execution chain. MPTCP's extra connection-management machinery gives the attacker more control over kernel allocations, which is what turns a use-after-free into reliable code execution.
- On a host **without** MPTCP, the reachable outcome on plain IPv4 TCP is a kernel crash (denial of service), kernel memory disclosure, or memory corruption. Red Hat's statement is that reliable code execution "requires precise heap control," which is the honest framing: the worst case is real but not trivial off the MPTCP path.

The fix

The fix is small and instructive (**Confirmed**). It changes the two offending frees, in `inet_sock_destruct()` and `tcp_v4_reqsk_destructor()` , from `kfree()` to `kfree_rcu()` , so the options structures are released only after an RCU grace period, by which time any reader that entered under `rcu_read_lock()` has finished. The rationale is simply that the readers already use RCU and other option-replacement paths already deferred their frees, so the destructors should too. The change landed upstream on 5 October 2026 and was backported across the maintained stable branches, from the 5.10 series through the current 7.x line. The CVE was published on 15 October 2026.

Timeline

Date (2026)	Event
5 October	Two-line `kfree_rcu` fix merged upstream and backported to maintained stable branches
15 October	CVE-2026-98432 published by the kernel CNA and to the GitHub Advisory Database (CVSS 9.8)
16 October	Oracle Linux UEK fixes released via ELSA advisories
28 October	Anthropic's Frontier Red Team publishes the AI-exploit report
30 October	AWS security bulletin published; Amazon Linux fixes available; Red Hat rates it Critical and is expediting fixes
2 November	Still no reported in-the-wild exploitation; not in CISA KEV

Part 2: the AI-written exploit, graded carefully

On 28 October 2026 Anthropic's Frontier Red Team published a post describing how an unreleased internal research model found and exploited memory-safety flaws in real software, and CVE-2026-98432 was its headline Linux example (**Reported**: Anthropic). The claim has a specific, checkable shape, which is why it is worth grading rather than repeating or waving away.

What the model did

Anthropic states that "after initial prompting, Claude wrote an exploit" that lets a remote attacker run commands as root, with no credentials or user interaction, on Linux servers with a Multipath TCP listener. The post attributes the work to "an unreleased, internal research model" and does not name it. The exploit it describes is a two-stage chain, which is the standard shape of a modern kernel exploit:

1. **Defeat KASLR.** Kernel Address Space Layout Randomization hides where kernel code and data sit in memory. The exploit first uses the use-after-free to leak a kernel pointer, which reveals the randomization offset and tells the attacker where everything is.
2. **Gain execution.** With the layout known, the attacker reuses the freed-and-reallocated object to redirect kernel control flow and reach a root shell.

By Anthropic's own account, "it takes 48 minutes to leak a kernel pointer and another five minutes to reach the shell command execution." That figure is Anthropic's measurement of its own run; it is not an independent benchmark.

What Anthropic withheld, and why

Anthropic reported the vulnerabilities under its coordinated disclosure policy, which it describes as giving developers 90 days to patch and a further 45 days before it discusses details beyond a public advisory. It did not publish the exploit code, the kernel version, or the tested configuration, and it framed the exercise as a lab result rather than an attack. Its stated purpose for publishing was defensive: that enabling defenders with capable models is "critical to giving them a head start on patching systems before attackers do," and that "scanning your own code is more important than ever." This context matters for grading: the omissions are deliberate and responsible, but they are also exactly what prevents independent verification.

What is missing for a higher grade

As of late October 2026 no independent party had publicly reproduced the exploit, and at least one kernel vendor team reported that it could not. Discovery credit for the specific CVE is also not cleanly stated: the post says "we discovered" a set of kernel findings and separately that "Claude wrote an exploit," without pinning which findings were model-found versus human-found, and the coordinated-disclosure report for this bug is associated with an Anthropic researcher. So the cleanest reading separates three questions:

- Is the vulnerability real and reachable? Yes, **Confirmed** by the kernel project and multiple vendors.
- Did an AI model write a working exploit for it in under an hour? **Reported** by Anthropic, unreleased and unreproduced.
- Did the AI discover the vulnerability in the first place? Not established either way by the public record.

Why it still matters at "Reported"

Grading the exploit **Reported** is not a dismissal. A capable lab saying, with a dated post and a real CVE number, that a model built a kernel remote-root exploit in under an hour is significant even before anyone reproduces it, because the direction it points is one defenders must plan around. Anthropic's own earlier assessment of a preview model described it chaining two, three and sometimes four vulnerabilities into working exploits against the Linux kernel, so this is a trend line, not a single anecdote. The responsible move for a defender is to act on the part that is already certain (patch), watch for reproduction, and avoid both the hype reading ("AI can now own any server") and the dismissive one ("vendor marketing"). See the AI Threats 2026 case study, whose machine-speed-vulnerability-discovery section this disclosure directly extends, and the AI Security Reference.

Part 3: patch status, mitigation and detection

The patch gap

The upstream fix existed before most organizations could deploy it, which is the usual kernel-CVE gap and the heart of the operational problem (**Confirmed**: distribution advisories, status as of late October to early November 2026):

Ecosystem	Status
Upstream stable	Fixed on 5 October 2026 and backported across maintained stable branches (5.10 series through current 7.x)
Debian 12 and 13	Fixed (DLA and DSA advisories issued)
SUSE SLES and openSUSE Leap	Fixed for most streams; some streams still in progress
Oracle Linux UEK	Fixed on 16 October 2026 via ELSA advisories
Amazon Linux 2 and 2023	Fixed kernel packages available; some default streams pending
AlmaLinux 8, 9, 10	Patched kernels released, after a period in testing
Red Hat Enterprise Linux 8, 9, 10 and OpenShift	Rated Critical (9.8); Red Hat was expediting fixes and evaluating interim OpenShift guidance
Rocky Linux 8 to 10 and other RHEL rebuilds	Followed RHEL; no fixed kernel at the time of writing
Ubuntu 22.04, 24.04, 26.04, 26.10	Fix in progress, not yet released

Per-platform remediation

The fix is always "run a patched kernel and reboot into it," but the command and the live-patch option differ by platform. Confirm the running kernel with `uname -r` after rebooting, not just the installed package.

Platform	How to check and remediate
Debian / Ubuntu	`uname -r` ; `sudo apt-get update && sudo apt-get install --only-upgrade linux-image-generic` (or your flavour's kernel meta-package), then reboot; Ubuntu Pro Livepatch may cover it before the reboot
RHEL / Alma / Rocky	`rpm -q kernel` ; `sudo dnf upgrade kernel` then reboot; where a live patch is published, `kpatch` can close it without a reboot (confirm coverage for this CVE)
Fedora	`sudo dnf upgrade kernel` and reboot
SUSE / openSUSE	`sudo zypper up kernel-default` then reboot; SLES Live Patching can bridge the reboot gap where a patch exists
Oracle Linux (UEK)	`sudo dnf upgrade kernel-uek` then reboot (UEK fixes shipped 16 October 2026); Ksplice can apply the fix live where available
Amazon Linux	`sudo dnf upgrade kernel` (AL2023) or `sudo yum update kernel` (AL2) then reboot; AL2023 instances pinned to a release need version 2023.12.20261102 or later
Arch	`sudo pacman -Syu linux` then reboot
Containers	Containers share the host kernel, so patch the node, not the image: a container cannot be patched for a kernel CVE. Cordon and drain the node, patch and reboot it, then uncordon
Managed Kubernetes	Roll node groups to a patched node image: EKS to a patched AL2023 AMI, GKE and AKS to a patched node image, draining nodes as they cycle
MPTCP mitigation	Find listeners with `ss` and check `sysctl net.mptcp.enabled` ; disabling MPTCP removes the reported remote-code-execution path but leaves the plain-TCP crash-and-corruption path, so it is a severity reduction, not a fix
Network mitigation	As a bridge, drop inbound IPv4 TCP packets carrying IP options at the host or edge (for example an nftables rule), after testing against legitimate traffic; it is not a substitute for patching

Two operational traps

First, a kernel fix is not live until the host reboots into the new kernel, so "package updated" is not the same as "no longer vulnerable" for a box that has not been restarted. For fleets that cannot reboot immediately, kernel live-patching (such as the live-patch services the major distributions offer) can close some classes of flaw

without a reboot, but live-patch coverage for any given CVE varies and should be confirmed with the vendor rather than assumed; it is a way to buy time, not a replacement for the eventual reboot.

Second, the common temporary mitigation, an nftables rule dropping inbound IPv4 TCP packets that carry IP options, is explicitly a bridge, not a fix: Red Hat and others say it does not replace patching, it needs testing against legitimate traffic (some networks use IP options), and on OpenShift nodes it covers only the host network namespace, not pod-to-pod traffic. Disabling Multipath TCP removes the reported remote-code-execution path but leaves the plain IPv4 TCP crash-and-corruption path reachable, so it reduces severity without closing the hole.

Detection and hunting

Because there is no reported in-the-wild activity, detection here is mostly about knowing your exposure and watching for the failure modes an unreliable exploit produces (**Confirmed** as general guidance; there are no published campaign indicators to match):

- **Inventory MPTCP listeners first.** These carry the remote-code-execution risk. Modern socket tooling can enumerate MPTCP sockets, and MPTCP is off by default on many distributions, so confirm whether it is enabled and listening before treating a host as worst-case.
- **Watch for kernel instability.** A failed or early-stage exploit against this bug tends to crash the kernel. A rise in kernel oops or panic events referencing the TCP or IPv4 networking paths on an internet-facing host is worth investigating, not just auto-restarting.
- **Flag unusual IP-options traffic.** Legitimate inbound IPv4 TCP with IP options is uncommon on most networks; a spike is a cheap signal, and it is the same traffic the temporary nftables mitigation drops.
- **Key on version and reboot, not just the installed package.** Confirm both that the fixed kernel package is installed and that the host is actually running it.

Part 4: the vulnerability-management lens

This CVE is a compact teaching case for how to triage in 2026, because its signals point in different directions and all of them are correct at once.

- **CVSS said "as bad as it gets."** A 9.8 with network access, no privileges and no user interaction is the maximum-urgency shape. On its own it argues for emergency patching everywhere.
- **EPSS and KEV said "not yet."** An EPSS around 0.46 percent and no KEV listing say that, despite the score and the headline, no one was observed using it. That argues against dropping everything, especially ahead of a lower-scored bug that is actively exploited.
- **Exposure said "only some of you."** The worst outcome needs an MPTCP listener; most hosts face denial of service, not remote code execution. Asset context changes the real risk more than the base score does.

The program move is to carry all three signals and let them combine, which is the core argument of the Vulnerability Prioritization Reference: fast-track this ahead of routine patching, scope the emergency to internet-facing and MPTCP-enabled hosts, use the network mitigation as a bridge where a reboot must wait, and do not let the AI-exploit headline reorder a queue that still has an actively exploited, lower-scored bug in it.

The AI angle changes one planning assumption, not the triage math: shorten the exploit-window your SLAs assume.

Cross-cutting themes

- **The exploit-availability window is the real clock.** Prioritization has always raced the moment a public exploit appears. This disclosure is a dated data point that, for memory-corruption bugs, that moment may now arrive within hours of the CVE, from an AI as readily as from a human researcher. Patch SLAs and emergency-change processes should be sized for that, not for the weeks exploit development used to take. See the Vulnerability Prioritization Reference.
- **Base severity and real-world risk diverged, and both were right.** A uniform 9.8 said the bug is as bad as bugs get in principle; the empty KEV entry and low EPSS said no one was using it yet; the MPTCP condition said the worst outcome only applied to some hosts. A program that collapses these into one number, in either direction, mis-triages. The job is to carry all three and act on their combination.
- **AI capability claims deserve the same evidence grading as everything else.** The right response to a vendor saying its model built a kernel exploit is neither to treat it as proven nor to dismiss it as marketing, but to grade it (single-source, unreproduced, Reported), act on the part that is already certain (the patch), and watch for reproduction. This is the same Confirmed-versus-Reported discipline the rest of the library applies to threat claims. See the AI Security Reference and Fuzzing and Vulnerability Research.
- **Defenders can run the same tools.** If a model can find and chain memory-safety bugs at this speed, the defensive implication Anthropic itself drew is that scanning your own code and dependencies before an advisory lands is now table stakes, not an advanced practice. The capability is dual-use; the question is who runs it first on your code.
- **Small fixes, large fleets.** The repair was two lines upstream, and the exposure still spanned the whole Linux ecosystem for weeks because of backporting, testing and the reboot requirement. Memory-safety classes like this one are also why the broader push toward memory-safe systems languages matters: the fix for the instance is quick, but the supply of the instances is the problem.

Sources

- The Linux kernel CVE record for CVE-2026-98432 and the GitHub Advisory Database entry GHSA-qp6h-4hvh-qqqc, including the root-cause description, the `kfree_rcu` fix, the CVSS 9.8 vector, the EPSS figure, and the IPv6 note.
- Red Hat's vulnerability page and security bulletin RHSB-2026-016 for CVE-2026-98432, for the Critical rating, the affected Red Hat Enterprise Linux and OpenShift versions, the "requires precise heap control" statement, and the nftables mitigation guidance.
- The AWS security bulletin for CVE-2026-98432, for the affected-version range, the Multipath TCP remote-code-execution statement, its statement that there was no evidence of exploitation in the wild, and the Amazon Linux fix status.
- Distribution advisories from Debian, SUSE, Oracle, AlmaLinux, Rocky Linux and Ubuntu, and vendor trackers, for the patch-status matrix.

- Anthropic's Frontier Red Team research post, "Discovering and exploiting vulnerabilities in critical, real-world software with Claude," for the exploit claim, the model description, the timing figures, the coordinated-disclosure framing, and the earlier preview-model assessment of multi-vulnerability chaining; the exploit, kernel version and configuration were not published, and the claim was not independently reproduced as of this writing.

Related references

- AI Security Reference
- MITRE ATLAS Reference
- Fuzzing and Vulnerability Research
- Vulnerability Prioritization Reference
- The AI Threats 2026 Case Study
- Notable Incidents, Vulnerability Management Reference